

RESEARCH ARTICLE OPEN ACCESS

Deep Reinforcement Learning-Based Control for Real-Time Hybrid Simulation of Civil Structures

Andrés Felipe Niño¹  | Alejandro Palacio-Betancur²  | Piedad Miranda-Chiquito³  | Juan David Amaya¹  | Christian E. Silva^{3,4}  | Mariantonieta Gutierrez Soto⁵  | Luis Felipe Giraldo⁶ 

¹Department of Electrical and Electronic Engineering, Universidad de los Andes, Bogotá, Colombia | ²Department of Civil and Environmental Engineering, The Pennsylvania State University, University Park, USA | ³Facultad de Ingeniería Mecánica y Ciencias de la Producción, Escuela Superior Politécnica del Litoral, ESPOL, Guayaquil, Ecuador | ⁴School of Mechanical Engineering, Purdue University, West Lafayette, USA | ⁵School of Engineering Design and Innovation, The Pennsylvania State University, University Park, USA | ⁶Department of Biomedical Engineering, Universidad de los Andes, Bogotá, Colombia

Correspondence: Mariantonieta Gutierrez Soto (mvg5899@psu.edu)

Received: 20 January 2024 | **Revised:** 5 October 2024 | **Accepted:** 2 January 2025

Funding: This study was supported by Corporación Ecuatoriana para el Desarrollo de la Investigación y la Academia (CEDIA) (CEPRA-XVI-2022-15-SIMULACIONHIBRIDA).

Keywords: data-driven control | neural network | real-time hybrid simulation | reinforcement learning | tracking control

ABSTRACT

Real-time Hybrid Simulation (RTHS) is a cyber-physical technique that studies the dynamic behavior of a system by combining physical and numerical components that are coupled through a boundary condition enforcer. In structural engineering, the numerical components are subjected to environmental loads that become dynamic displacements of the physical substructure applied through an actuator. However, the dynamics of the coupling between components and the complexities of the system lead to synchronization challenges that affect the accuracy of the simulation. Thus, requiring tracking controllers to ensure the fidelity of the simulation. This paper studies deep reinforcement learning (DRL) as a novel alternative to designing the tracking controller. Three controllers are designed: a DRL agent combined with a conventional time delay compensation, a conventional feedback controller combined with a DRL agent, and a DRL agent with a complex neural network architecture. The proposed approaches are tested using a virtual RTHS benchmark problem, and the results are compared with an optimized controller that has a proportional-integral-derivative controller and phase-lead compensation. The results show that DRL can address the synchronization challenges of RTHS with a model-free approach and simple neural network architectures. The work shown in this study is a critical step toward model-free control methodologies that can transform and further develop the RTHS method. The proposed methodology can be used to address important challenges related to RTHS, including nonlinearities and uncertainties of the physical substructure, complex boundary conditions, and the computational efficiency when physical structures with complex dynamics are present.

1 | Introduction

Evaluating the dynamic performance of civil structures is critical to understanding their structural dynamics. Large-scale testing of buildings and bridges poses financial and practical challenges.

To overcome these challenges, researchers use real-time hybrid simulation (RTHS) as an attractive alternative. Hybrid simulation (HS) is a combined experimental technique in which the tested system is partitioned into numerical and physical components, which are coupled in real-time [1] through a boundary

Abbreviations: DRL, deep reinforcement learning; HS, hybrid simulation; RTHS, real-time hybrid simulation.

[Correction added on 01 February 2025, after first online publication: Author Christian E. Silva affiliations have been corrected in this version.]

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial](https://creativecommons.org/licenses/by-nc/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

© 2025 The Author(s). *International Journal of Robust and Nonlinear Control* published by John Wiley & Sons Ltd.

condition enforcer called a transfer system. In other words, a part of the system is simulated computationally, the other is tested physically, and both are linked together so that it acts as a single system. After this partitioning is established, the numerical component is subjected to an external disturbance that generates a physical response to the physical component. The dynamic response is measured and fed back into the numerical portion to calculate the next response step. When this process occurs in real-time for structures that are rate-dependent, the method is known as RTHS. The early stages of HS are reported in [2] and show how this method was proposed for the first time in 1969 [1], which developed into real-time simulation [3]. This method allows researchers to evaluate novel systems at low costs because the partitioning can be designed so that elements with complex dynamics are tested physically in existing lab facilities and the numerical components are based heavily on computational power.

Furthermore, certain systems are too complex to be modeled mathematically without large uncertainties but are also too large or too expensive to be tested physically. In these cases, RTHS proves to be a middle ground to obtain realistic results without having to investigate physically at a large scale [4]. Also, RTHS is a technique that can be applied in the study of structural control, as it is not practical to test for response-induced damages on large-scale structures for every design. Consequently, using RTHS is a powerful tool that researchers can use as an alternative to generating realistic experimental results. A realistic RTHS test is achieved when the interface between numerical and physical components is moving simultaneously. However, the transfer system has time-dependent dynamics that introduce a time delay in the system [4]. This synchronization problem compromises the accuracy of the simulation. Thus, a tracking controller is required to follow the reference given by the numerical partition. A diagram that illustrates the tracking problem is shown in Figure 1. Therefore, control methodologies for tracking displacement and other features have been on the rise. The first controller for RTHS was the polynomial extrapolation proposed by Horiuchi et al. [5] that predicts the target displacement using a determined number of previous samples. This approach has been implemented as the basis of more robust compensation methodologies [6, 7]. In RTHS, the system has several sources of delays as indicated by Carrion and Spencer [8] which are the computation time, D/A and A/D conversions, communication time, the inherent dynamics of the actuator, and the controller sampling frequency. However, which is attributed the longest desynchronization time is the one produced by the actuator dynamics. Zhao

et al. [9] implemented the Phase-Lead Compensator to improve amplitude and phase errors. That said, the real-time tests aim to stabilize the delay with a compensation; therefore, several methods have been proposed to minimize the delay, which is based on an inverse compensation (IC) [10], which subsequently offer greater robustness and adaptability, such as an adaptive IC (AIC) and an improved AIC technique [11, 12] with estimates at each time step. Another compensation method based on error indicators in the frequency domain was proposed in [13] to estimate in line at each window change. Another common controller is the model-based compensation proposed by Carrion and Spencer [14], it uses the characteristics of the physical substructure to design a feed-forward controller, and it has also been used in robust controllers [15, 16]. Chen and Ricles [17] developed the inverse compensation that uses a first-order model of the transfer system to predict the target displacement. Lately, the development of tracking controllers has included adaptive algorithms to take into account variable time delays that may occur in physical components. These include the adaptive polynomial extrapolation [18, 19], adaptive time series (ATS) compensation [20–22], adaptive model-based [23–25], and sliding mode control [26]. A state-of-the-art review of control methodologies for RTHS can be found in [27] and multi-actuator considerations in [28].

The majority of the mentioned control methodologies require a model of physical components to tune parameters and guarantee the accuracy and stability of the simulation. For this reason, researchers are exploring data-driven methods to estimate models and design adaptive compensation algorithms without the knowledge of the physical components. Simpsons, Dertimanis and Chatzi [29] expressed the plant as a discrete finite impulse response model and used it to design an adaptive inverse compensation controller without prior knowledge of the system dynamics. Yu et al. [30] proposed an adaptive Kriging approach to take into account time-dependent and independent uncertainties of physical components. Moreover, data-driven methods like Reinforcement learning (RL) are also being implemented in control engineering. RL is a machine learning technique that trains an agent based on rewards while it interacts with an environment. This has been implemented to solve multiple problems in photovoltaic systems [31], turbofan engines [32], water distribution systems [33], and smart structures [34], and failure prediction of aircraft maintenance [35].

Traditional RL algorithms often rely on tabular methods or function approximation techniques that use handcrafted features to represent value functions or policies. Policy Iteration (PI) and

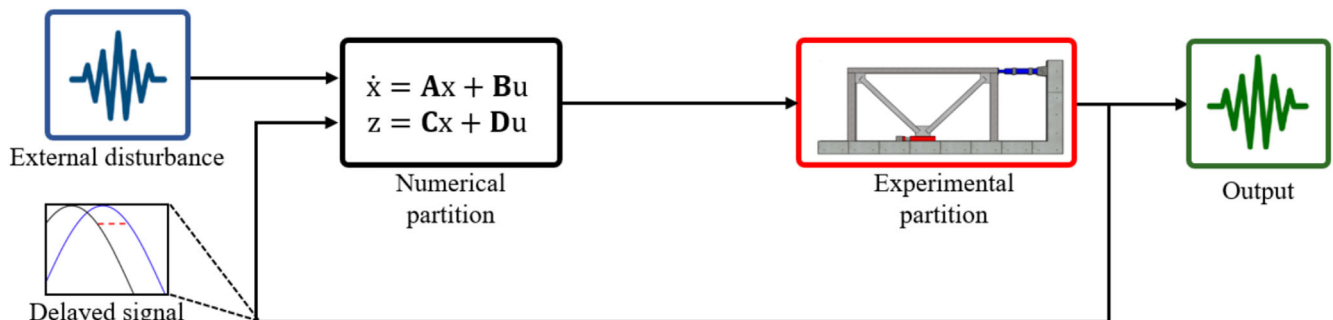


FIGURE 1 | Tracking control problem introduced due to time delay in RTHS simulations.

Value Iteration (VI) are examples of these algorithms, since they converge to an optimal policy or optimal value function, respectively, in a discrete action/state tabular space to solve a problem [36]. Although these methods might work well for small state and action spaces, they struggle with scalability to large and continuous spaces. Therefore, the concept of RL is also combined with artificial deep neural networks to approximate value functions or policies. This is known as Deep Reinforcement Learning (DRL), whose objective is to produce an agent that interacts with an environment to learn optimal behaviors [37] as shown in Figure 2. In DRL, these neural networks are capable of automatically learning features from raw sensory inputs, like in this case the time series from an RTHS dynamical system, enabling them to handle high-dimensional state and action spaces more effectively. This approach allows the design of a controller with a model-free approach, which is ideal for complex systems with unknown mathematical models that usually require sophisticated controllers and tuning of parameters. Li et al. [38] proposed the first RL-based tracking controller for RTHS based on a deep deterministic policy gradient algorithm combining an agent as a feedback controller and a conventional feed-forward time delay compensation. The controller performance was evaluated with the vRTHS benchmark problem and an underwater shaking table test control.

The contribution of this paper is to provide a methodology for the design of DRL-based controllers to solve the RTHS tracking control problem. The objective is to produce agents that send the required control signal to the physical partition so that it follows the reference of the numerical partition with the minimum error possible. Three different controller architectures are explored: a DRL agent combined with a conventional time delay compensation, a conventional feedback controller combined with a DRL agent, and a single DRL agent with a complex neural network. The performance of the proposed techniques is evaluated using a virtual RTHS (vRTHS) benchmark problem of a three-story steel building with a one-bay, one-story steel frame as a physical substructure. The accuracy and robustness of the DRL agents are discussed in detail and compared with an optimized controller that has a proportional-integral-derivative (PID) controller and a phase lead.

2 | Benchmark Problem

To test the use of reinforcement learning to solve the tracking control problem in RTHS, we will use the benchmark problem proposed by Silva et al. [4]. This particular benchmark has been studied to test the effectiveness of control methodologies such as Linear-Quadratic-Gaussian control [7], sliding mode control

[39, 40], backstepping adaptive control [41], adaptive model predictive control [42], model-based compensation [43] and adaptive tracking control based on a conditional adaptive time series compensation [21]. A state-of-the-art review of control methods and a comprehensive comparative study between conventional and adaptive control methods using the benchmark problem are reported in [27].

The vRTHS consists of a three-story steel moment-resisting frame where the physical substructure is one bay, one-story moment-resisting frame attached to an actuator, and the numerical substructure is the remaining beams and columns of the building as shown in Figure 3. Thus, our proposed controller's goal is to generate the necessary actuator displacement so that the moment-resisting frame has the same displacement as the first

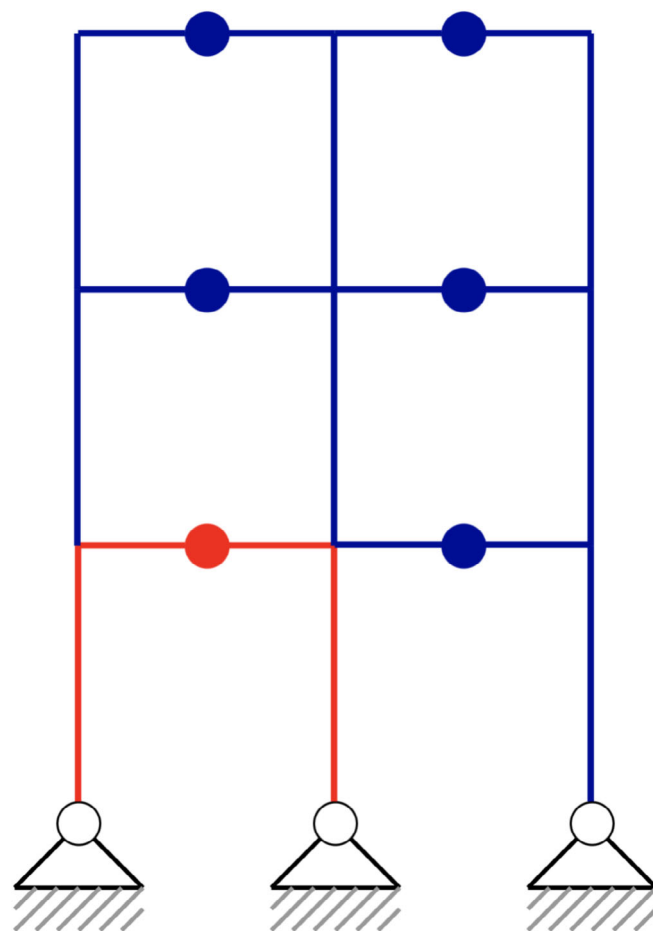


FIGURE 3 | Partition of the structure, where the portion highlighted in red is the experimental substructure (adapted from Silva et al. [4]).

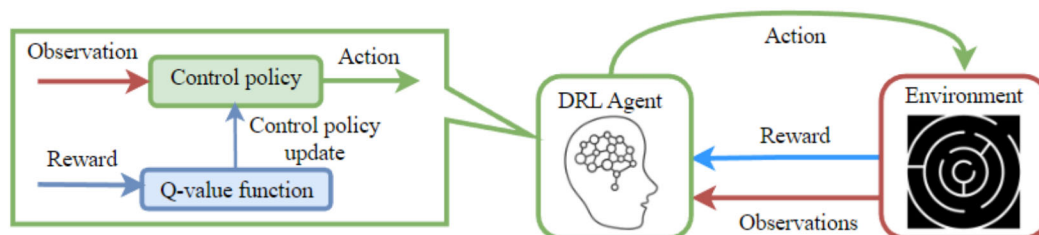


FIGURE 2 | Diagram of DRL agent in a control loop.

floor of the numerical substructure. In addition, this study will implement the first partitioned case consisting of 1000 kg mass on each story and reference modal damping of 5%.

The block diagram of the vRTHS is shown in Figure 4. The external disturbance and the force feedback (f_e) are the input to the numerical substructure; y_n is the output of the numerical substructure that contains displacements, velocities, and accelerations for all three stories that can be used as one of the inputs to the tracking controller (G_c). The tracking controller produces the compensated command signal (y_{gc}) to the physical substructure known as the control plant (G_p). The output of the physical substructure is measured and fed back to the corresponding blocks.

The external disturbance used in this study is the El Centro historical earthquake scaled down to 40% intensity, and the linear model of the plant is given by:

$$G_p(s) = \frac{B_0}{A_5 s^5 + A_4 s^4 + A_3 s^3 + A_2 s^2 + A_1 s + A_0} \quad (1)$$

where

$$\begin{aligned} B_0 &= a_1 \beta_0 \\ A_0 &= k_e a_3 \beta_2 + a_1 \beta_0 \\ A_1 &= k_e a_3 \beta_1 + (k_e + c_e a_3 + a_2) \beta_2 \\ A_2 &= k_e a_3 + (k_e + c_e a_3 + a_2) \beta_1 + (c_e + m_e a_3) \beta_2 \\ A_3 &= (k_e + c_e a_3 + a_2) + (c_e + m_e a_3) \beta_1 + m_e \beta_2 \\ A_4 &= c_e + m_e a_3 + m_e \beta_1 \\ A_5 &= m_e \end{aligned} \quad (2)$$

The values of all the parameters are shown in Table 1.

The results of the simulation are then compared to a reference response, which is the response of the system as if it was not partitioned (that is, the whole structure being simulated numerically). The performance parameters that will be used to evaluate any of the designed controllers are listed in Table 2. The first indicator is the only one not associated with relative error measures, but with time delay. Specifically, the time delay between the command and tracked displacement of the actuator. The mathematical expression of J_1 represents the cross-correlation between the desired and measured displacements, represented by the number of discrete simulation time steps that the measured response is away from maximum correlation (zero time delay). When multiplying this value with the sampling frequency, one obtains the resulting time-delay in milliseconds.

The discrete-time controller given as an example in the vRTHS benchmark problem consists of a Proportional-Integral (PI) feedback controller and a Phase-Lead (PL) compensator to reduce the time delay as shown in Figure 5. The sample feedback controller was designed using a Ziegler-Nichols step response method that yields the gains $k_p = 2$ and $k_i = 95$. Then, the pole-zero pair of the PL compensator is chosen to improve rise time, settling time, and overshoot. For the sample controller, the parameters are $z_1 = 168.6$, $p_1 = 8570$, and $k_T = 50.8$.

However, the sample controller yields evaluation criteria above 10% and time delay above 4 ms, which are relatively high values for the benchmark problem [27]. For this reason, the parameters of the sample controller are optimized using tools from MATLAB to have better results as a baseline for the DRL controllers. First,

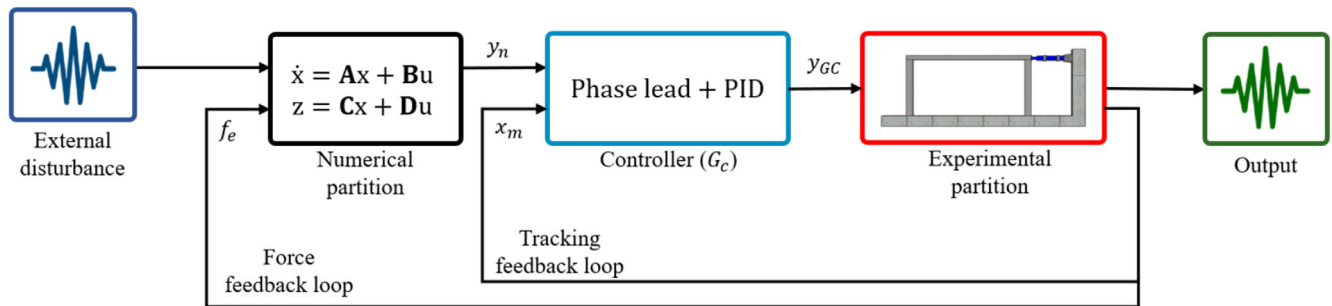


FIGURE 4 | VRTHS block diagram.

TABLE 1 | Parametric values of the plant as stated in Silva et al. [4].

Parameter	Component	Nominal Value	Standard deviation	Units
$a_1 \beta_0$	Servo-valve	2.13×10^{13}	—	mPa/s
a_2	CSI	4.23×10^6	—	mPa
a_3	Actuator	3.3	1.3	1/s
β_1	Servo-valve	425	3.3	—
β_2	Servo-valve	10×10^4	3.3×10^3	1/s
m_e	Exp. mass	29.12	—	kg
k_e	Exp. stiff.	1.19×10^6	5×10^4	N/m
c_e	Exp. damp.	114.6	—	kg/s

TABLE 2 | Performance evaluation criteria of the benchmark problem [4].

Criterion	Equation	Units
Time delay	$J_1 = \arg\max_k \left[\sum_i y_n^{(1)}(i) x_m(i - k) \right]$	ms
Tracking error NRMS	$J_2 = \sqrt{\frac{\sum_{i=1}^N [x_m(i) - y_n^{(1)}(i)]^2}{\sum_{i=1}^N [y_n^{(1)}(i)]^2}} \times 100$	%
Peak tracking error	$J_3 = \frac{\max x_m(i) - y_n^{(1)}(i) }{\max y_n^{(1)}(i) } \times 100$	%
1 st floor NRMS	$J_4 = \sqrt{\frac{\sum_{i=1}^N [x_m(i) - x_r^{(1)}(i)]^2}{\sum_{i=1}^N [x_r^{(1)}(i)]^2}} \times 100$	%
1 st floor peak error	$J_5 = \frac{\max x_m(i) - x_r^{(1)}(i) }{\max x_r^{(1)}(i) } \times 100$	%
2 nd floor NRMS	$J_6 = \sqrt{\frac{\sum_{i=1}^N [y_n^{(2)}(i) - x_r^{(2)}(i)]^2}{\sum_{i=1}^N [x_r^{(2)}(i)]^2}} \times 100$	%
2 nd floor peak error	$J_7 = \frac{\max y_n^{(2)}(i) - x_r^{(2)}(i) }{\max x_r^{(2)}(i) } \times 100$	%
3 rd floor NRMS	$J_8 = \sqrt{\frac{\sum_{i=1}^N [y_n^{(3)}(i) - x_r^{(3)}(i)]^2}{\sum_{i=1}^N [x_r^{(3)}(i)]^2}} \times 100$	%
3 rd floor peak error	$J_9 = \frac{\max y_n^{(3)}(i) - x_r^{(3)}(i) }{\max x_r^{(3)}(i) } \times 100$	%

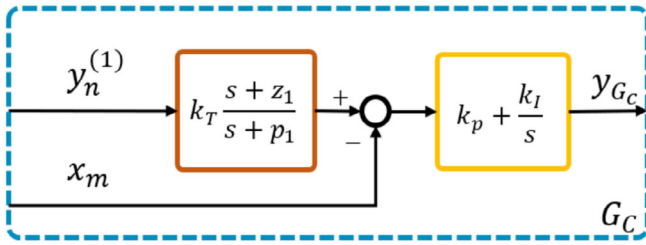


FIGURE 5 | Sample controller structure designed by the authors of the benchmark.

the PI is tuned using the Root Locus Editor that yields $k_p = 1.52$ and $k_i = 60$. Then, the pole-zero pair of the PL compensator is computed with the same methodology as the sample controller that yields $z_1 = 64.15$, $p_1 = 22520$, and $k_T = 351.08$.

Therefore, this paper presents an AI-based optimization approach to the baseline PI of the benchmark control problem. Given the fact that PID adjustment is a challenging problem, particularly in the initial stages, where the experimental plant can be driven to instability, we propose this optimization technique as a computational alternative to well-established methodologies such as Ziegler-Nichols. The intended goal is to provide a safer, faster, and more optimal initial tuning procedure for a minimum delay tracking performance of the benchmark problem in RTHS.

3 | Deep Reinforcement Learning for Control in RTHS

The way a Deep Reinforcement Learning agent operates within a control loop is shown in Figure 2. Intuitively, the way the agent is trained can be explained as follows: The environment is in a state at time step i , denoted as s_i . The state refers to the current value of the variables the agent is controlling, such as the position of an autonomous car or the jaw, pitch, and roll of a drone. This state is received by the agent, and based on the control policy it has, an action A_i is taken to change the agent's state in the environment,

which is denoted as s_{i+1} . Said action also generates a reward R_{i+1} , which could be described as a numerical value that states how far the current environment state is from the desired state [44]. Based on this reward, the agent updates its control policy, and based on this new policy, it takes a new action A_{i+1} to modify its state once again. This process continues until the last time step of the episode.

An agent updates its action policy by using a Q -value function, denoted as $v_\pi(s_i)$. This function estimates the expected reward from following policy π starting from the current state s_i until the end of the episode [45]. During training, the agent aims to maximize the Q -value function by exploring actions and finding the optimal policy. Typically, the gradient of the Q -value function is used to update the policy, as it serves as an indication of whether a local maximum has been achieved or not [37].

The control scheme we propose to use DRL on an RTHS problem is shown in Figure 6. The agent receives the outputs of both the numerical and physical partition, and based on that, it generates the observations used to compute the action, as well as the rewards used for the Q -value function. We expect that the use of DRL will introduce a framework to design a controller within the RTHS framework since it is a model-free method to obtain control policies. This means that a mathematical model is not required to obtain a control policy [44], which is ideal for complex systems or when different assumptions introduce uncertainties to the model. Consequently, the agent learns by trial and error [46], which means it explores different policies until it can perform the optimal control action.

While the current benchmark may not fully exploit the capabilities of DRL, the experimentation presented in this paper with these techniques allows us to assess the performance and efficiency in a controlled setting. Establishing a foundation in DRL-based approaches early on provides the foundation to efficiently explore and exploit the benefits of these techniques as it encounters more challenging problems in the future in the field of RTHS.

3.1 | DRL Optimal Policy

The reward function determines how far an action is from its objective; therefore, it is a critical component of the DRL problem because the maximization problem to be solved during training depends on it [47]. A reward function can be either a reward for the agent (how much an agent can progress in a video game level) or punishment (the time taken to swing a pendulum to a desired position) [44]. In this case, since it is a tracking problem, the function used to calculate the Q -values will be a punishment. This means the output of the reward function will be a negative value, and the training process will make this value as close to 0 as it is possible. Then, the reward function is defined as:

$$R[e(i), s_i] = -[E(e(i)) + P(s_i)] \quad (3)$$

The first term (E) is a function of the tracking error, and the second term (P) is a function of undesirable states, for instance, when the limits of an actuator are exceeded or when the derivative of certain observations is too high.

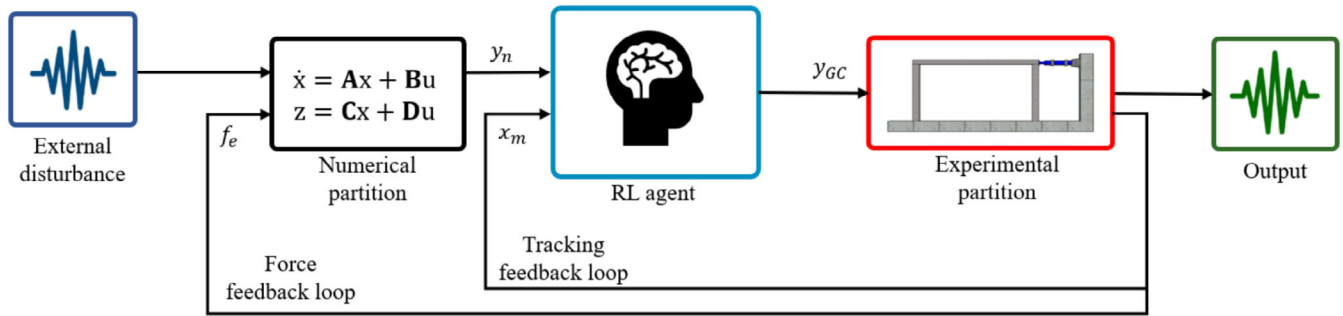


FIGURE 6 | Diagram of DRL agent in the tracking control problem introduced by RTHS.

3.2 | DRL Algorithm

DRL algorithms can be used according to the environment and required actions. Actuators typically have large and continuous actions; consequently, the Reinforcement Learning algorithm to be used must support continuous action spaces. A popular algorithm to design controllers in feedback control systems is the Actor-Critic algorithm known as Deep Deterministic Policy Gradient (DDPG). DDPG is specifically designed for problems with continuous action spaces, making it well-suited for tasks where precise and continuous control is required.

An Actor-Critic agent can be divided into two separate parts. The actor, which stores the control policy (the function that maps observations to actions), and the Critic, calculate the expected reward based on the current state [37]. In other words, the critic can be understood as the Q-value estimation. During training, the agent learns both the control policy and the Q-value estimation. Since DDPG is a *Deep* Reinforcement Learning algorithm, both the actor and the critic consist of two different neural networks. The actor and critic networks are updated every time step based on the Q-value estimation and the states of the system. The stability of these networks is increased by applying a smoothing factor (τ) to previous time steps. Also, the DDPG agents use *batch learning*, which means that the agent stores state transitions from s_i, A_i to s_{i+1}, A_{i+1} and then samples from this buffer to train during every time step. This makes training independent of the size of the time step and reduces the variance in weight updates [37].

3.3 | Exploration Vs Exploitation Dilemma

One of the main problems in reinforcement learning lies in knowing when the agent must explore or exploit [37]. Exploration makes references to taking random actions to find those that are better than others. A broader action space helps the agent find an optimal policy according to the reward of each action. Conversely, exploitation occurs when the agent takes actions that yield a better reward. Excessive exploration will cause the agent to never find an optimal policy because it will never exploit the actions that have better rewards. Whereas excessive exploitation leads to a lack of exploration and the agent will possibly converge to a non-global maximum. Therefore, the exploration of the continuous action space of the DDPG agent is encouraged by adding white noise to the output of the actor network during training [37].

The parameters associated with the proposed DDPG agent are the discount rate (γ), which states how much the Q-value function takes future rewards into account [45], the gradient threshold that defines the maximum value of the gradient of the Q-value function that updates the weights of the neural networks [48], and the learning rate that establishes how much the weights change based on the gradient-based updates from the Q-value function [49].

3.4 | Design Approaches of DRL for Vtrhs

The primary goal of this research is to develop an agent capable of effectively serving as a tracking controller for Real-Time Hybrid Simulation (RTHS), as illustrated in Figure 6. This entails the execution of feedback control and time delay compensation. While tuning the environment for various Reinforcement Learning (RL) problems remains somewhat of a trial-and-error process [46], our approach in this study utilizes informed strategies for designing neural networks. We begin with simple network architectures and progressively explore more complex ones. The selection of RL architectures is driven by our aim to create methodologies applicable to RTHS tracking control. Specifically, we choose architectures that can be viewed as extensions of the linear controllers proposed in previous literature for this benchmark.

In particular, the first approach consists of a DRL agent for feedback control and a PL compensator that will be titled DRL1, which is studied with a simple architecture (DRL1.1) and a complex architecture (DRL1.2). The second approach has a PI controller for feedback control and a DRL agent for time delay compensation that will be referenced as DRL2. Then, the last approach consists of a single DRL agent for feedback control and time delay compensation named DRL3. The details of each approach are explained below.

3.4.1 | DRL 1.1: Using a Simple DRL Agent as the Feedback Controller

The first DRL design approach consists of an agent to perform feedback control and a phase-lead compensator to reduce time delay in the command signal using the parameters from the sample controller. The architecture of the DRL agent is a neural network equivalent to a PI controller as shown in Figure 7. The inputs to the network are the error and its integral. The error is obtained with the following expression:

$$e(i) = |y_p(i) - x_m(i)| \quad (4)$$

where $y_p(i)$ is the compensated displacement of the first story and $x_m(i)$ is the measured displacement from the experimental substructure, both at time step i . These values will be connected to a single hidden layer, and the weights of the inputs will be the proportional and integral constants of a PI controller. This means we are essentially tuning the PI controller using a DRL agent system.

The hidden layer internally adds both inputs and a bias and then applies an activation function that generates the output. To make the network equivalent to the operation made by a PI controller, the bias is set to zero. The activation function used will be a leakyReLU layer. Since the goal of the DRL-based approach to design a controller is to learn an agent that can reduce $e(i)$, the reward function for this design approach, from Equation (3), becomes:

$$R_i = -K[e(i)]^2 \quad (5)$$

The squared factor of the error penalizes higher values of errors, and the value of the gain K is set to 1000 because the errors have an order of magnitude of 10^{-4} . In addition, the hyper parameters for the RL problem are shown in Table 3, and the parameters of the phase-lead compensator are the same as the sample controller.

3.4.2 | DRL 1.2: Using a Complex DRL Agent as the Feedback Controller

The previous DRL design approach can be improved by using a more complex neural network to get a better control policy. Using an excessive amount of neurons and layers may result in the neural network not converging, and it has been shown that DRL rarely needs more than two hidden layers [50]. Therefore, the new neural network will have two additional hidden layers, compared

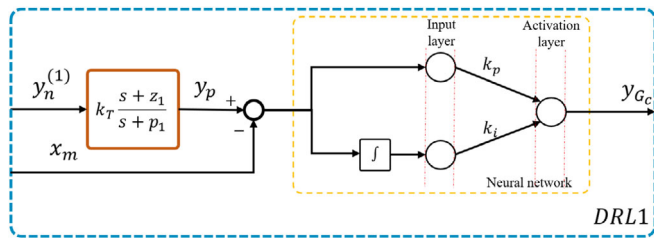


FIGURE 7 | Architecture of DRL1 equivalent to a PI controller.

to DRL1.1, with two neurons each, and using the hyperbolic activation function. These functions may generate actions outside of the actuator's range of motion; thus, a scaling layer is required. This layer consists of two hidden layers, a hyperbolic tangent (tanh) layer that adds the outputs, and a second layer that scales the value to the actuator's operating range of ± 7 mm.

The reward function for this DRL agent is, from Equation (3), as follows:

$$R_i = \begin{cases} -(K_1 e(i)^2 + K_2 A_{i-1}^2 + K_3) & \text{if } [A_i - A_{i-1}]^2 > 2 \times 10^{-3} \\ -(K_1 e(i)^2 + K_2 A_{i-1}^2) & \text{if } [A_i - A_{i-1}]^2 < 2 \times 10^{-3} \end{cases} \quad (6)$$

This function uses the error from Equation (4), and it has two additional terms to orient the agent towards a desired behavior. The actuator actions (A_i) are taken into account to penalize continued large actions and strong oscillations. Thus, the new neural network will be stable. The gains used for this reward function are $K_1 = 2 \times 10^5$, $K_2 = 2500$, and $K_3 = 25000$. The hyper-parameters for DRL1.2 are shown in Table 3, and the parameters of the phase lead compensator are also the same as the sample controller.

3.4.3 | DRL 2: Using a DRL Agent as the Feed-Forward Controller

The second DRL design approach for vRTHS consists of the PI feedback controller from the sample controller and a DRL agent to reduce the time delay in the command signal. The proposed architecture of the neural network is shown in Figure 8, and it is similar to the one used for the PI neural network. However, the inputs are the computed displacement of the first story from the numerical substructure ($y_n^{(1)}(i)$) and the increase from the previous displacement ($\Delta y_n^{(1)}(i) = y_n^{(1)}(i) - y_n^{(1)}(i-1)$).

The reward function for this agent is expressed in Equation (5); the gain is increased to $K = 2 \times 10^5$ because the error is expected to be smaller in magnitude, and the error is obtained as follows:

$$e(i) = |y_n^{(1)}(i) - x_m(i)| \quad (7)$$

The hyper-parameters for DRL2 are shown in Table 3, and the parameters of the PI controller are the same as the sample controller.

Note that this neural network architecture in DRL2 is specifically crafted to utilize historical information of $y_n^{(1)}$ as input. Through

TABLE 3 | Hyper-parameters for DRL tracking controllers for RTHS. The discount factor was chosen by trial and error.

Parameter	DRL1.1	DRL1.2	DRL2	DRL3
Experience buffer length	10^6	10^6	10^6	10^6
Mini-batch size	512	1024	1024	2048
Learn rate	10^{-4}	10^{-4}	$10 - 4$	5×10^{-5}
Discount factor	0.99	0.99	0.99	0.99
Gradient threshold	1	1	1	0.7
Noise standard deviation	0.1	0.005	0.0025	[0.005, 0]
Noise mean	—	—	—	[0, 0]
Noise decay standard deviation	—	—	—	3.3845×10^{-6}

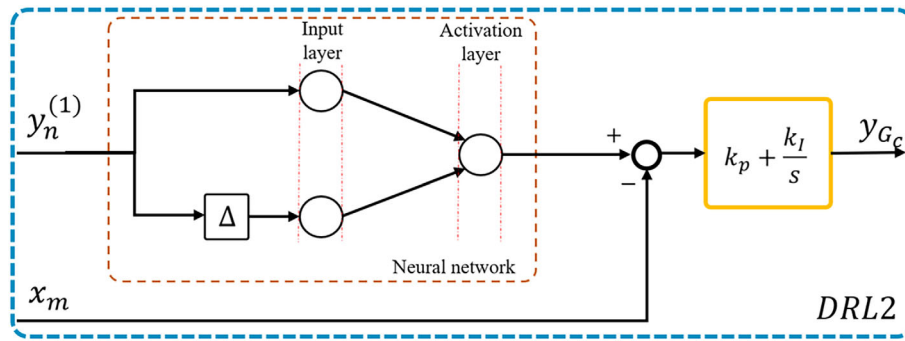


FIGURE 8 | Architecture of DRL2 replacing the phase lead compensation.

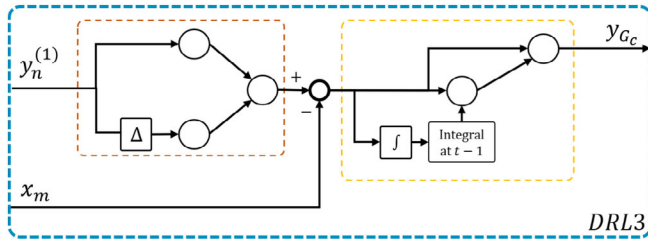


FIGURE 9 | Integrated neural network architecture of DRL3.

a learning process, the network aims to minimize control error, functioning as a time-forecasting model. By successfully anticipating computed displacement responses, it intended to act as a compensatory system within the control framework.

3.4.4 | DRL 3: Complete Tracking Controller for RTHS Using a Single DRL Agent

The last DRL design approach integrates the feedback controller and the time delay compensation in a single DRL action policy based on the results obtained in the previous approaches. The proposed neural network is shown in Figure 9.

The time delay compensation layer has the same architecture as DRL2 using the computed displacement from the numerical substructure. Then, the architecture of DRL1 is replicated using the output of the first layer and the measurement from the physical substructure. To calculate the integral of the error at time step i , it was necessary to add an extra layer that combines the error at time step i with the integral at a previous time step $i - 1$.

The reward function used for this DRL agent is the same as the one used in Equation (5) with a gain $K = 2.5 \times 10^5$, where the error is taken as the tracking error of the system obtained with Equation (7). The tuning of the hyper parameter of this complex architecture required several iterations. At first, the weights and bias of the time-delay and feedback control layers were used as learnable parameters, but in the end, it was only necessary to use the weights as learnable parameters. The starting point for the tuning process was using the hyper parameters used for DRL1 and DRL2. However, the agent's learning was slower than in previous approaches. Therefore, the following decisions were made:

- **Activation Function:** The neural network architecture of the agent is made up of different convolutional layers to

achieve the desired policy. One of these layers is the activation function *leaky ReLU* after each layer containing one of the training weights of the network.

- **Weight_INITIALIZER:** It is important to decide how to initialize weights to avoid symmetry problems, vanishing gradients, or exploding gradients. Initially, the *He* and *Xavier* methods were considered, but the agent did not perform well compared to DRL1 or DRL2. Thus, the weights were initialized manually with values close to the optimized sample controller from Section 2.
- **Experience Buffer Length:** The *buffer length* or *replay buffer* is the vector that stores the transition states that have been used for training. If the size of this buffer is comparable to the size of the states obtained in a training session, there is no negative effect in the learning process. Since it has been observed that the training converges in several steps smaller than the size of the replay buffer of 10^6 , it was decided to use this value.
- **Mini-Batch Size:** The gradients for the neural networks are computed using a random selection of transition states from the experience buffer. It was decided to avoid very large values, and after trial and error, the mini-batch size is chosen as 2048.
- **Batch Normalization:** This is used to allow higher values of global learning rate without divergence due to the normalization of inputs to the neural network. However, the approach of modeling a PI controller with DDPG does not perform normalization because it removes the units of the variables.
- **Global Learning Rate:** There are different tuning methods to find an optimal value, including the cyclical learning rates [51]. However, this method was not necessary because the values used for DRL1 and DRL2 of 5×10^{-5} proved to be effective in training the agent.
- **Noise Process:** It was decided to use the Ornstein-Uhlenbeck distribution to generate a temporally correlated exploration; this allows an efficient exploration in physical systems with inertia that require control [52].
- **Noise Mean:** The behavior of the Ornstein-Uhlenbeck noise distribution is governed by several parameters. One of them is the noise mean that defines the center of the signal. Since

the agent's control action signal needs to be centered at the value 0, then the noise mean is defined as 0.

- **Noise Standard Deviation:** This parameter for the noise distribution is the standard deviation that defines the fluctuation of the signal. For the agent output representing the tracking error, a standard deviation of zero is used since the exact error value is needed for the neural network architecture. As for the control action, it was decided to use the value of 0.005.
- **Noise Standard Deviation Decay Rate:** This parameter defines the rate at which the noise standard deviation value decreases at each step and is formulated as:

$$\#steps = \frac{\log(0.5)}{\log(1 - D)} \quad (8)$$

After sensitivity analyses, $D = 3.3845 \times 10^{-6}$ makes the standard deviation decay by half in seven episodes. In addition, it was observed that this affects the convergence speed of the training.

After the tuning process was done, the agent ended up with the hyper parameters shown in Table 3.

4 | Analysis and Simulations

4.1 | Virtual RTHS Results

The training process of the proposed DRL agents was described in the previous Section 3. The hyperparameters are included in Table 3, and the reward function for each episode during the training of the DRL agents is shown in Figure 10. The discount factor was chosen by trial and error. A saturation term was applied to constrain the actuators within the range of -0.01 to 0.01 . These limits were chosen because they were an order of magnitude higher than the actuator values in the benchmark case. Then, the benchmark vRTHS problem is used to simulate the El Centro historical earthquake scaled by a factor of 0.4 with the first partitioned case, and the displacement results for the optimized sample controller and the DRL agents are shown in Figure 11. Furthermore, the corresponding performance evaluation criteria for all methods are shown in Table 4.

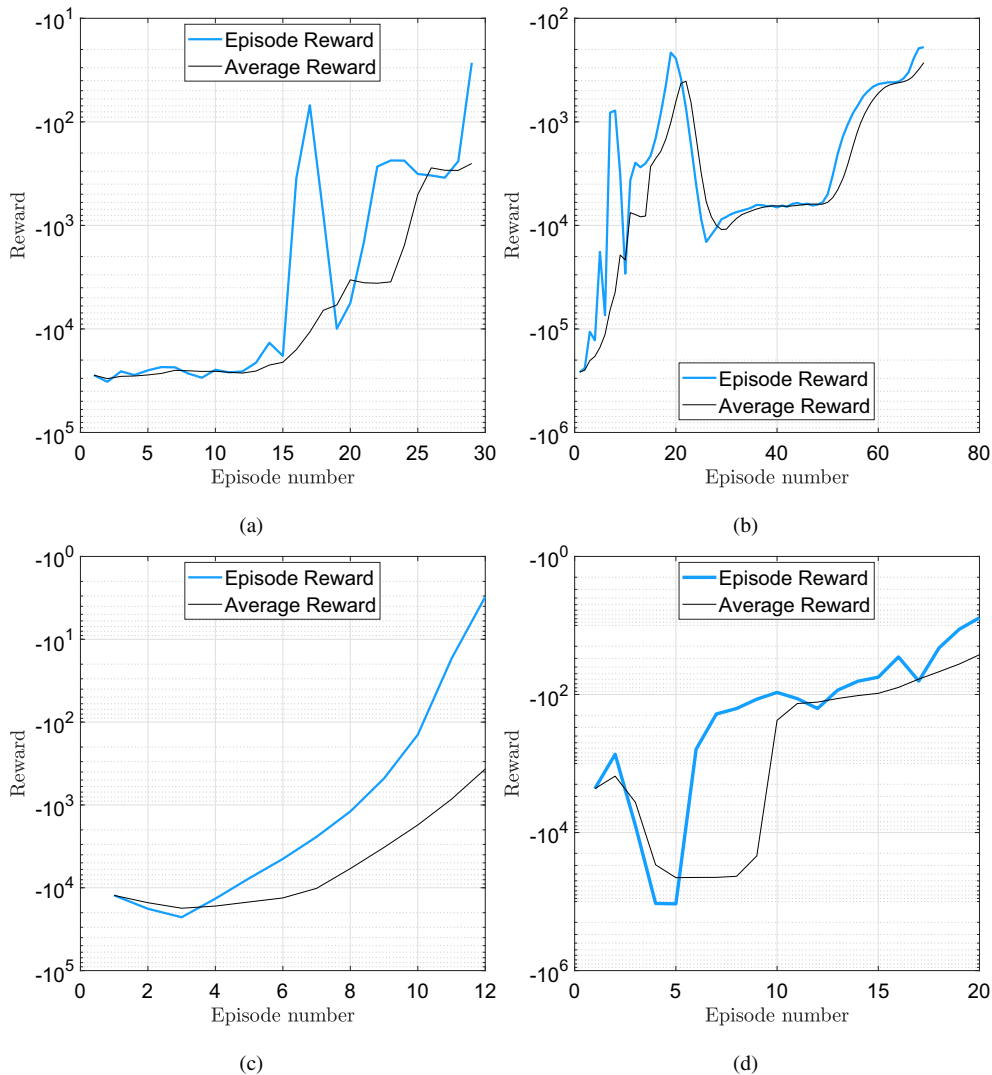


FIGURE 10 | Reward function for each episode during training for (a) DRL1.1; (b) DRL1.2; (c) DRL2; and (d) DRL3.

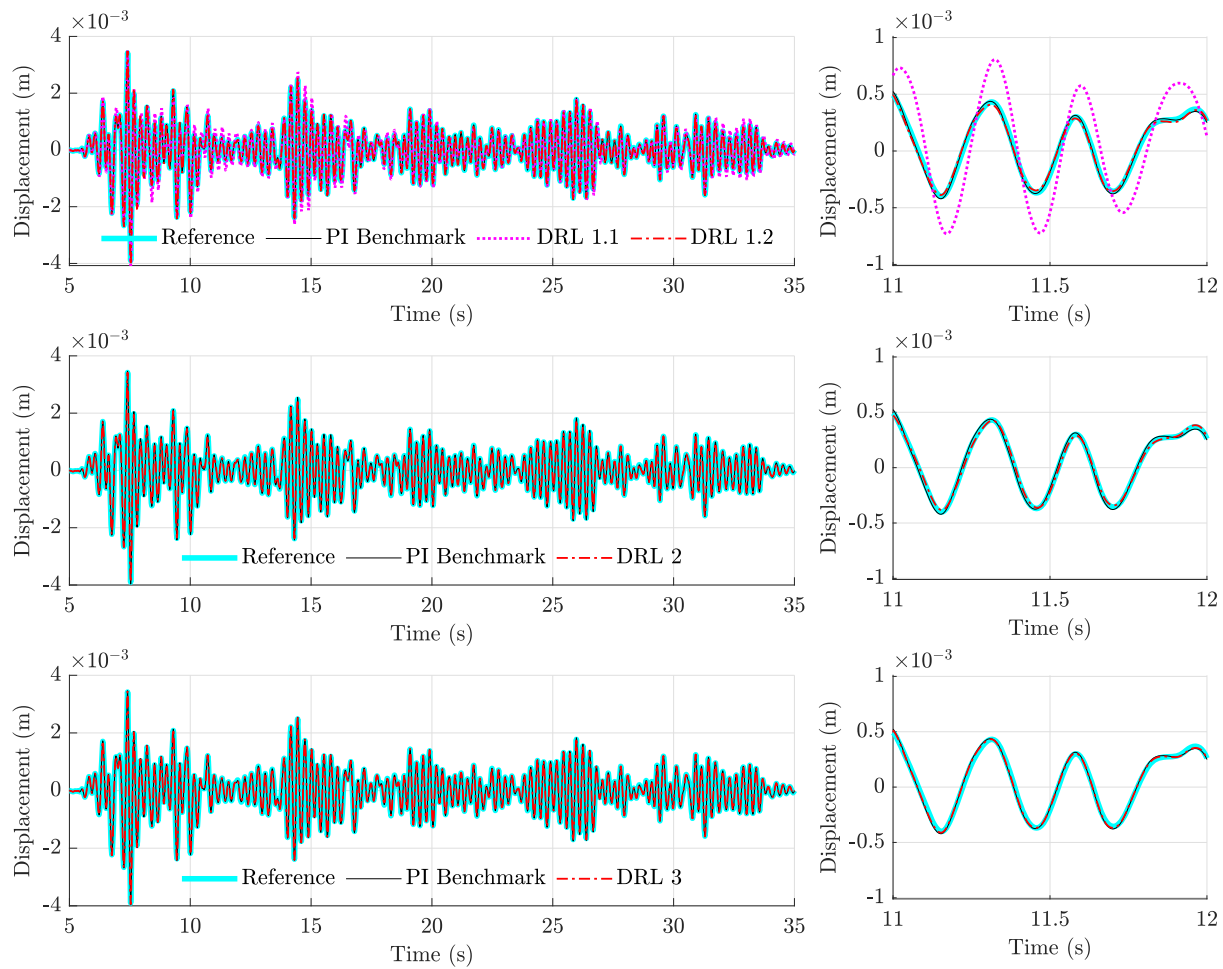


FIGURE 11 | Accuracy assessment of DRL agents for a 0.4 scaled El Centro earthquake. Left: Full time scales; Right: Zoomed sections. (The time scale was cut for better visualization).

TABLE 4 | Comparison of the results obtained with various control approaches. The **bold** denotes the best performance for the selected criterion.

Criterion	PI+PL	Optimized PI+PL	DRL1.1	DRL1.2	DRL2	DRL3
J_1 (ms)	4.6	0.2	3.2	0.2	0	0.2
J_2 (%)	10.4	2.72	14.21	4.47	3.21	2.71
J_3 (%)	11.5	2.38	15.02	11.99	3.45	2.41
J_4 (%)	36.1	4.73	35.47	6.93	3.71	4.19
J_5 (%)	19.0	3.34	18.98	15.00	3.62	3.11
J_6 (%)	32.3	3.58	33.73	4.52	1.78	2.81
J_7 (%)	32.4	3.59	33.74	4.55	1.78	2.82
J_8 (%)	14.8	1.93	18.31	4.98	0.95	1.53
J_9 (%)	14.8	1.93	18.16	5.12	0.99	1.50

The behavior of the reward function for DRL1.1 shows that the proposed noise for exploration yields good results because some episodes increase the reward function considerably. These actions are then exploited to converge to a reward function value close to -30 , a similar value to the reward obtained with the sample controller. This DRL design approach emulates a PI controller, and it was able to reduce the time delay to 3.2 ms. However, it does not handle measurement noise well in the feedback

loop because all the other performance criteria are similar to or above the values of the sample controller. This behavior is visible in Figure 11, especially in lower amplitudes, because the tracking controller does not follow the reference response.

On the other hand, the DRL1.2 agent, which also replaces the PI feedback controller, was able to reduce the time delay to 0.2 ms, and the other performance criteria were considerably reduced

as well. It means that his trained agent can handle measurement noise better than the DRL1.1; nevertheless, it still has large error values at higher peaks, as evidenced by J_3 and J_5 , and on high amplitudes shown in Figure 11. It is important to note that these peak errors occur at the highest intensity of the earthquake; therefore, the training of the agents was not able to handle sudden changes in the input command. The behavior of the reward function during training for this agent shows how the reduced exploration led to better exploitation of some actions in the first episodes. However, after 20 episodes of getting closer to a desired value of -100 , the reward decreased. A possible cause for this is a phenomenon known as *catastrophic forgetting*, which occurs when an agent forgets past experiences as it loses old information to incorporate new experiences relevant to the current task [53]. Once additional training episodes are carried out, the agent finds better actions to exploit and reaches the reward threshold by episode 69.

As a comparison, DRL2 only required 12 episodes to reach a desired value of -5 , but it is probably related to the different definition of the error used in the reward function. The implementation of a lower standard deviation for the additive Gaussian noise also helped with a faster convergence because the agent could exploit better actions from the start of the training process. The most important observation for this agent is that the time delay was reduced to zero and every performance criterion is under 4%. This is an important improvement compared to previous agents, and it solves the issue of sudden changes reflected in J_3 and J_5 and in Figure 11. The parameters related to the tracking error (J_2 and J_3) are slightly higher than the optimized sample controller, which means that the measurement noise from the experimental substructure still affects performance.

Finally, the DRL3 agent took 20 episodes to converge to a value of -7 while using an error function similar to DRL2. This shows that the chosen hyper parameters led to a good exploration and exploitation during training. The time delay using this design approach was reduced to 0.2 ms and all the other performance criteria do not exceed 4.5%. This agent also solves the issue of sudden changes as shown in Figure 11. A last important observation is that this agent handles measurement noise better than DRL2, with values close to the optimized sample controller.

4.2 | Robustness Analysis

In addition to the criteria assessment, it is necessary to carry out robustness tests to determine the performance of the proposed controllers under different random conditions. For this purpose, the Optimized PI controller and the DRL3 are chosen and then subjected to ten simulations with perturbed parameters subjected to the El Centro historical earthquake for the first partition case.

To compare the different results obtained from the robustness tests, we proceed to calculate the mean and standard deviation of all the evaluation criteria obtained in each case to better visualize the comparison between both controllers. The obtained values are shown in Figure 12.

It can be seen that in most cases the DRL controller presents lower values for both the mean and the standard deviation of

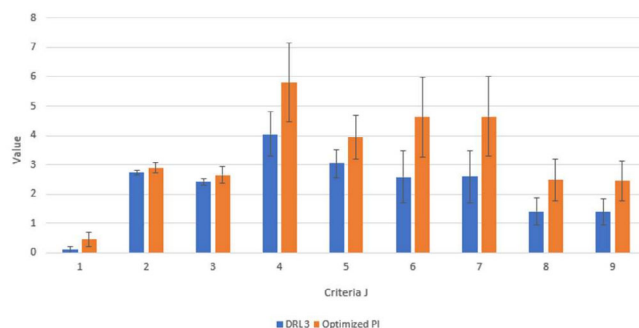


FIGURE 12 | Robustness comparison between the optimized PI controller and DRL3.

the evaluation criteria than the Optimized PI controller. Furthermore, in no case was the capacity of the hydraulic actuator exceeded [4]. Therefore, the DRL3 tracking controller is the most robust.

5 | Conclusions

RTHS has the potential to yield more realistic and useful results with lower costs and computational power. However, the execution in real-time of these simulations poses a challenge because the inherent dynamics of the experimental system introduce a time delay that may compromise the performance of the simulation. This study shows that DRL can be used to design a tracking controller that compensates for this added time delay with a model-free approach. Our results show that, when the optimization problem behind the training process of the DRL agent is correctly tuned, an optimal solution can be achieved, even with simple neural network architectures. The results also show how the DRL model can be adapted to fulfill different roles within the tracking controller, as shown by the three design approaches of the agents. Thus, a DRL model can be adapted for multiple RTHS environments without needing a complex controller design approach. It is also possible to integrate multiple portions of the controller within the same control policy, which reduces the necessary steps to design an RTHS environment.

Future developments in this area should focus on integrating feedback control and time delay compensation in a single control policy. Once that is achieved, the research could focus on increasing the complexity of the neural networks, as well as testing the DRL approach on other RTHS environments with different time delays and performance requirements. For instance, in the vRTHS benchmark problem, there are multiple partitioning cases with different time delays that impact the performance of the DRL agents. It is also important to use different DRL algorithms to find which one yields a better controller in the least possible amount of training time. Additionally, DRL can be studied for RTHS that considers multiple experimental substructure configurations and determines optimal computational resources [54]. Future research includes evaluating the proposed DRL methodology for real-time hybrid shake table testing at high-performance experimental facilities and studying the methodology's performance for nonlinear systems such as frames during a collapse. Future directions should consider

studying the proposed methods for multi-axial RTHS in the newly released benchmark problem by [55] that challenges the research community with controlling two hydraulic actuators as the transfer system that aims to create the desired translation and rotation at the interface [22].

Our study has identified the robust performance of the DRL-based controller under changes in the dynamical system as a significant advantage. However, it is essential to acknowledge that our research was conducted in a relatively simple simulated environment. This controlled setting may not fully capture the complexity and unpredictability present in real-world scenarios. A strategy to evaluate the robustness of the DRL model is through adversarial testing in more complex scenarios, such as the ones described before, where the model is subjected to extreme or unforeseen conditions to assess its performance under stress. By systematically introducing disturbances or perturbations into the simulation environment, we can evaluate the model's ability to adapt and recover from unexpected events. Adversarial testing provides valuable insights into the model's vulnerabilities and helps identify improvement areas, as shown in recent work in structural control systems [56]. Furthermore, to enhance the generalization capabilities of the DRL model across different environments, transfer learning and domain adaptation techniques can be explored. By pre-training the model on a diverse set of simulated environments and fine-tuning it on target environments, we can leverage knowledge transfer to improve performance and adaptability in unpredictable scenarios. Additionally, domain adaptation methods can help bridge the gap between simulated and real-world data distributions, enabling the model to better generalize to novel environments.

The work shown in this study is an important step toward model-free control methodologies that can transform and further develop the RTHS method. The proposed methodology can be used to address important challenges related to RTHS, including nonlinearities and uncertainties of the physical substructure, complex boundary conditions, and the computational efficiency when physical structures with complex dynamics are present.

Nomenclature

β_1	Parameter of the transfer system associated with the servo-valve
β_2	Parameter of the transfer system associated with the servo-valve
γ	DDPG discount rate
τ	Smoothing factor
$a_1\beta_0$	Parameter of the transfer system associated with the servo-valve
A	State matrix of the numerical substructure
B	Input matrix of the numerical substructure
C	Output matrix of the numerical substructure
D	Feedthrough matrix of the numerical substructure
x	States
y	Outputs
a_2	Parameter of the transfer system associated with the actuator
a_3	Parameter of the transfer system associated with the actuator

A_i	RL action i
A_l	Dummy variable for the coefficients of the transfer function of the plant
c_e	Damping coefficient of the experimental substructure
$E()$	Tracking error function
$e(t)$	Error function
J_i	i th Performance evaluation index
K_i	Reward function gain i
k_i	PID integral gain
k_p	PID proportional gain
k_T	PL gain
k_e	Stiffness of the experimental substructure
m_e	Mass of the experimental substructure
$P()$	Undesirable states function
p_i	i pole location
$R()$	Reward function
R_i	RL reward i
s	Laplace operator
x_m	Measured feedback displacement (for tracking)
y_n	Numerical substructure output displacement
y_p	Compensated displacement 1st story
y_{GC}	Command signal (controller output) from controller to transfer system
z_i	i zero location
f_e	Measured feedback force (for RTHS loop)
ReLU	Rectified linear unit
ATS	Adaptive time series
CSI	Control-structure interaction
DRL	Deep deterministic policy gradient
DRL	Deep reinforcement learning
FE	Finite element
HS	Hybrid simulation
ML	Machine learning
PID	Proportional-integral-derivative
PL	Phase-lead
RL	Reinforcement learning
RTHS	Real-time hybrid simulation
v-RTHS	Virtual real-time hybrid simulation

Acknowledgments

Partial support for this work was provided by CEDIA under the CEPRA XVI Program at Escuela Superior Politécnica del Litoral (ESPOL), which is kindly acknowledged.

Disclosure

The authors have nothing to report.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in DesignSafe-CI at <https://doi.org/10.17603/ds2-9nqb-b991>, reference number PRJ-3554.

References

1. M. Hakuno, M. Shidawara, and T. Hara, "Dynamic Destructive Test of a Cantilever Beam, Controlled by an Analog-Computer," in *Proceedings of the Japan Society of Civil Engineers*, vol. 1969 (Japan: Japan Society of Civil Engineers, 1969), 1–9.
2. M. Nakashima, "Hybrid Simulation: An Early History," *Earthquake Engineering and Structural Dynamics* 49, no. 10 (2020): 949–962.
3. M. Nakashima, H. Kato, and E. Takaoka, "Development of Real-Time Pseudo Dynamic Testing," *Earthquake Engineering and Structural Dynamics* 21, no. 1 (1992): 79–92.
4. C. E. Silva, D. Gomez, A. Maghareh, S. J. Dyke, and B. F. Spencer, Jr., "Benchmark Control Problem for Real-Time Hybrid Simulation," *Mechanical Systems and Signal Processing* 135 (2020): 106381.
5. T. Horiuchi, M. Inoue, T. Konno, and Y. Namita, "Real-Time Hybrid Experimental System With Actuator Delay Compensation and Its Application to a Piping System With Energy Absorber," *Earthquake Engineering and Structural Dynamics* 28, no. 10 (1999): 1121–1141.
6. X. Ning, Z. Wang, H. Zhou, B. Wu, Y. Ding, and B. Xu, "Robust Actuator Dynamics Compensation Method for Real-Time Hybrid Simulation," *Mechanical Systems and Signal Processing* 131 (2019): 49–70.
7. H. Zhou, D. Xu, X. Shao, X. Ning, and T. Wang, "A Robust Linear-Quadratic-Gaussian Controller for the Real-Time Hybrid Simulation on a Benchmark Problem," *Mechanical Systems and Signal Processing* 133 (2019): 106260.
8. J. E. Carrion and B. F. Spencer, *Model-Based Strategies for Real-Time Hybrid Testing* (Champaign, IL: Newmark Structural Engineering Laboratory report series, University of Illinois at Urbana-Champaign, 2007).
9. J. Zhao, C. French, C. Shield, and T. Posbergh, "Considerations for the Development of Real-Time Dynamic Testing Using Servo-Hydraulic Actuation," *Earthquake Engineering and Structural Dynamics* 32, no. 11 (2003): 1773–1794.
10. C. Chen, *Development and Numerical Simulation of Hybrid Effective Force Testing Method* (Bethlehem, PA: PhD Dissertation, Department of Civil and Environmental Engineering, Lehigh University, 2007).
11. C. Chen, J. M. Ricles, R. Sause, and R. Christenson, "Experimental Evaluation of an Adaptive Inverse Compensation Technique for Real-Time Simulation of a Large-Scale Magneto-Rheological Fluid Damper," *Smart Materials and Structures* 19, no. 2 (2010): 025017, <https://doi.org/10.1088/0964-1726/19/2/025017>.
12. C. Chen, J. M. Ricles, and T. Guo, "Improved Adaptive Inverse Compensation Technique for Real-Time Hybrid Simulation," *Journal of Engineering Mechanics* 138, no. 12 (2012): 1432–1446, [https://doi.org/10.1061/\(ASCE\)EM.1943-7889.0000450](https://doi.org/10.1061/(ASCE)EM.1943-7889.0000450).
13. W. Xu, T. Guo, and C. Chen, "Localized Evaluation of Actuator Tracking for Real-Time Hybrid Simulation Using Frequency-Domain Indices," *Structural Engineering and Mechanics* 62, no. 5 (2017): 631–642, <https://doi.org/10.12989/sem.2017.62.5.631>.
14. J. E. Carrion, B. Spencer, and B. M. Phillips, "Real-Time Hybrid Simulation for Structural Control Performance Assessment," *Earthquake Engineering and Engineering Vibration* 8, no. 4 (2009): 481–492.
15. X. Gao, N. Castaneda, and S. J. Dyke, "Real Time Hybrid Simulation: From Dynamic System, Motion Control to Experimental Error," *Earthquake Engineering and Structural Dynamics* 42, no. 6 (2013): 815–832.
16. G. Ou, A. I. Ozdagli, S. J. Dyke, and B. Wu, "Robust Integrated Actuator Control: Experimental Verification and Real-Time Hybrid-Simulation Implementation," *Earthquake Engineering and Structural Dynamics* 44, no. 3 (2015): 441–460.
17. C. Chen and J. M. Ricles, "Development of Direct Integration Algorithms for Structural Dynamics Using Discrete Control Theory," *Journal of Engineering Mechanics* 134, no. 8 (2008): 676–683.
18. A. Darby, M. Williams, and A. Blakeborough, "Stability and Delay Compensation for Real-Time Substructure Testing," *Journal of Engineering Mechanics* 128, no. 12 (2002): 1276–1284.
19. D. Xu, H. Zhou, X. Shao, and T. Wang, "Performance Study of Sliding Mode Controller With Improved Adaptive Polynomial-Based Forward Prediction," *Mechanical Systems and Signal Processing* 133 (2019): 106263.
20. Y. Chae, K. Kazemibidokhti, and J. M. Ricles, "Adaptive Time Series Compensator for Delay Compensation of Servo-Hydraulic Actuator Systems for Real-Time Hybrid Simulation," *Earthquake Engineering and Structural Dynamics* 42, no. 11 (2013): 1697–1715.
21. A. Palacio-Betancur and M. Gutierrez Soto, "Adaptive Tracking Control for Real-Time Hybrid Simulation of Structures Subjected to Seismic Loading," *Mechanical Systems and Signal Processing* 134 (2019): 106345, <https://doi.org/10.1016/j.ymssp.2019.106345>.
22. A. J. Aguila, H. Li, A. Palacio-Betancur, K. A. Ahmed, I. Kovalenko, and M. Gutierrez Soto, "Conditional Adaptive Time Series Compensation and Control Design for Multi-Axial Real-Time Hybrid Simulation," *Frontiers in Built Environment* 10 (2024): 1384235.
23. P. C. Chen, C. M. Chang, B. F. Spencer, and K. C. Tsai, "Adaptive Model-Based Tracking Control for Real-Time Hybrid Simulation," *Bulletin of Earthquake Engineering* 13, no. 6 (2015): 1633–1653.
24. C. Gálmez and G. Fermandois, "Robust Adaptive Model-Based Compensator for the Real-Time Hybrid Simulation Benchmark," *Structural Control and Health Monitoring* 9, no. 7 (2022): e2962, <https://doi.org/10.1002/stc.2962>.
25. A. Najafi and B. F. Spencer, Jr., "Adaptive Model Reference Control Method for Real-Time Hybrid Simulation," *Mechanical Systems and Signal Processing* 132 (2019): 183–193, <https://doi.org/10.1016/j.ymssp.2019.06.023>.
26. A. Maghareh, S. J. Dyke, and C. E. Silva, "A Self-Tuning Robust Control System for Nonlinear Real-Time Hybrid Simulation," *Earthquake Engineering and Structural Dynamics* 49, no. 7 (2020): 695–715.
27. A. Palacio-Betancur and M. Gutierrez Soto, "Recent Advances in Computational Methodologies for Real-Time Hybrid Simulation of Engineering Structures," *Archives of Computational Methods in Engineering* 30 (2022): 1–26.
28. A. Najafi, G. Ferandois, S. Dyke, and B. Spencer, "Hybrid Simulation With Multiple Actuators: A State-Of-The-Art Review," *Engineering Structures* 2 (2023): 276. Publisher Copyright: © 2022 Elsevier Ltd, <https://doi.org/10.1016/j.engstruct.2022.115284>.
29. T. Simpson, V. K. Dertimanis, and E. N. Chatzi, "Towards Data-Driven Real-Time Hybrid Simulation: Adaptive Modeling of Control Plants," *Frontiers in Built Environment* 6 (2020): 570947.
30. G. Yu, C. Chen, H. Hou, C. Peng, and R. Zhang, "Integrating Adaptive Kriging With Expansion Optimal Linear Estimation Into Real-Time Hybrid Simulation for Time-Variant Experimental Analysis of Structures With Deterioration," *Journal of Building Engineering* 64 (2023): 105658.
31. J. F. Gaviria, G. Narváez, C. Guillen, L. F. Giraldo, and M. Bressan, "Machine Learning in Photovoltaic Systems: A Review," *Renewable Energy* 196 (2022): 298–318.
32. Y. Tian, M. A. Chao, C. Kulkarni, K. Goebel, and O. Fink, "Real-Time Model Calibration With Deep Reinforcement Learning," *Mechanical Systems and Signal Processing* 165 (2022): 108284.

33. J. Martinez-Piazuelo, D. E. Ochoa, N. Quijano, and L. F. Giraldo, "A Multi-Critic Reinforcement Learning Method: An Application to Multi-Tank Water Systems," *IEEE Access* 8 (2020): 173227–173238.
34. S. Javadinasab Hormozabad, N. Jacobs, and M. Gutierrez Soto, "Reinforcement Learning for Integrated Structural Control and Health Monitoring," *Practice Periodical on Structural Design and Construction* 29, no. 3 (2024): 04024026.
35. M. D. Dangut, I. K. Jennions, S. King, and Z. Skaf, "Application of Deep Reinforcement Learning for Extremely Rare Failure Prediction in Aircraft Maintenance," *Mechanical Systems and Signal Processing* 171 (2022): 108873.
36. W. Gao, M. Mynuddin, D. C. Wunsch, and Z. P. Jiang, "Reinforcement Learning-Based Cooperative Optimal Output Regulation via Distributed Adaptive Internal Model," *IEEE Transactions on Neural Networks and Learning Systems* 33, no. 10 (2021): 5229–5240.
37. K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A Brief Survey of Deep Reinforcement Learning," 2017. *arXiv preprint arXiv:1708.05866*.
38. N. Li, J. Tang, Z. X. Li, and X. Gao, "Reinforcement Learning Control Method for Real-Time Hybrid Simulation Based on Deep Deterministic Policy Gradient Algorithm," *Structural Control and Health Monitoring* 29, no. 10 (2022): e3035.
39. H. Li, A. Maghareh, H. Montoya, J. W. C. Uribe, S. J. Dyke, and Z. Xu, "Sliding Mode Control Design for the Benchmark Problem in Real-Time Hybrid Simulation," *Mechanical Systems and Signal Processing* 151 (2021): 107364.
40. K. D. Young, V. I. Utkin, and U. Ozguner, "A Control engineer's Guide to Sliding Mode Control," *IEEE Transactions on Control Systems Technology* 7, no. 3 (1999): 328–342.
41. Y. Ouyang, W. Shi, J. Shan, and B. F. Spencer, "Backstepping Adaptive Control for Real-Time Hybrid Simulation Including Servo-Hydraulic Dynamics," *Mechanical Systems and Signal Processing* 130 (2019): 732–754.
42. N. Tsokanas, R. Pastorino, and B. Stojadinović, "Adaptive Model Predictive Control for Actuation Dynamics Compensation in Real-Time Hybrid Simulation," *Mechanism and Machine Theory* 172 (2022): 104817.
43. G. A. Fernando, "Application of Model-Based Compensation Methods to Real-Time Hybrid Simulation Benchmark," *Mechanical Systems and Signal Processing* 131 (2019): 394–416.
44. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* ch. Introduction (Cambridge, MA: MIT Press, 2018), 1–13.
45. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* ch. Finite Markov Decision Processes (Cambridge, MA: MIT Press, 2018), 47–68.
46. Z. Ding and H. Dong, *Deep Reinforcement Learning* (Singapore: Springer, 2020).
47. M. A. Arroyo and L. F. Giraldo, "Data-Driven Outer Loop Control Using Deep Reinforcement Learning for Trajectory Tracking," 2020. *arXiv preprint arXiv:2008.13732*.
48. R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proceedings of the 30th International Conference on Machine Learning, in Proceedings of Machine Learning Research*. (Cambridge, UK: Proceedings of Machine Learning Research, 2013).
49. E. Schuitema, L. Buşoni, R. Babuška, and P. Jonker, "Control Delay in Reinforcement Learning for Real-Time Dynamic Systems: A Memoryless Approach," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems* (New York: Institute of Electrical and Electronics Engineers IEEE, 2010), 3226–3231.
50. J. Heaton, *Introduction to Neural Networks with Java, 2nd Edition Feedforward Neural Networks*, 2nd ed. (St. Louis, MO: Heaton Research, Inc, 2008).
51. L. N. Smith, "Cyclical Learning Rates for Training Neural Networks," in *2017 IEEE Winter Conference on Applications of Computer Vision* (New York: Institute of Electrical and Electronics Engineers IEEE, 2017), 464–472.
52. T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al., "Continuous Control With Deep Reinforcement Learning," 2016 arXiv:1509.02971.
53. J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al., "Overcoming Catastrophic Forgetting in Neural Networks," 2017 arXiv:1612.00796.
54. J. P. Waldbjoern, A. Maghareh, G. Ou, S. J. Dyke, and H. Stang, "Multi-Rate Real Time Hybrid Simulation Operated on a Flexible LabVIEW Real-Time Platform," *Engineering Structures* 239 (2021): 112308.
55. J. W. Condori Uribe, M. Salmeron, E. Patino, et al., "Experimental Benchmark Control Problem for Multi-Axial Real-Time Hybrid Simulation," *Frontiers in Built Environment* 9 (2023): 1270996.
56. A. Zambrano, A. Palacio-Betancur, L. Burbano, et al., "You Make Me Tremble: A First Look at Attacks Against Structural Control Systems," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (New York: Association for Computing Machinery, 2021), 1320–1337.